

Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Trabalho Prático: Fase 4

Computação Gráfica

Ano Letivo 2025/2026

Grupo 03

A106919 Eduardo Freitas Fernandes

A106842 Gonçalo Rodrigues Ribeiro

A106888 José Mário Raimundo Lima

CG

Índice

1. Introdução	2
2. Organização do Código	2
3. Arquitetura do Sistema	2
4. Ficheiro .3d	3
5. <i>Generator</i>	3
5.1. Plano	3
5.2. Cubo	4
5.3. Esfera	4
5.4. Cone	4
5.4.1. Base	4
5.4.2. Superfície Lateral	4
5.5. Cilindro	5
5.5.1. Tampas	5
5.5.2. Superfície Lateral	5
5.6. Anel	5
5.7. <i>Bezier Patches</i>	6
6. <i>Engine</i>	7
6.1. Iluminação	7
6.2. Materiais	7
6.3. Texturas	8
7. <i>Demo Scene</i>	8
8. Conclusão	9

1. Introdução

Esta fase acrescenta ao sistema desenvolvido nas fases anteriores suporte a iluminação, materiais e texturas. O *generator* foi atualizado para calcular e exportar normais e coordenadas de textura para todas as primitivas. O *engine* passou a configurar fontes de luz OpenGL, a aplicar propriedades de material por modelo e a carregar imagens para texturas através da biblioteca *stb_image*. O formato do ficheiro *.3d* foi estendido para conter, por vértice, posição, normal e coordenadas UV. As funcionalidades das fases anteriores (VBOs/IBOs, animações, *Bézier patches*) mantêm-se inalteradas.

2. Organização do Código

A estrutura do projeto mantém a divisão em *packages* introduzida na fase 3:

- *generator/*: geração de primitivas geométricas, agora com normais e coordenadas UV.
- *engine/*: *parsing* do XML, gestão da cena e ciclo principal.
- *lib/core/*: estruturas de dados partilhadas (*Model*, *Point*, *Vector*). A classe *Model* foi estendida com as estruturas *Material* e UV, e com lógica de carregamento de texturas.
- *lib/stb_image/*: inclusão da biblioteca *stb_image* para leitura de ficheiros de imagem.
- *lib/xml/*: dependência *tinycl2*.

3. Arquitetura do Sistema

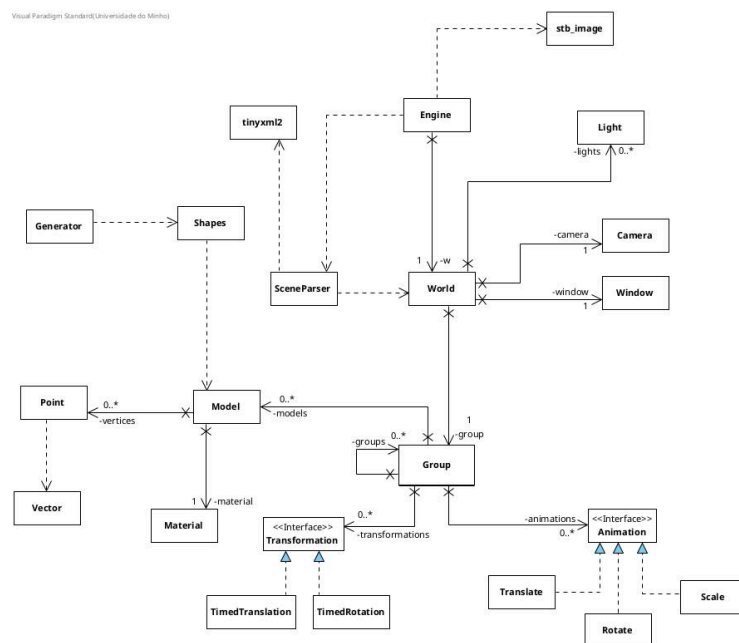


Figura 1: Arquitetura do Sistema

A classe *World* passa a manter uma lista de instâncias de *Light* e a chamar *Light::enableLighting()* durante a inicialização e *Light::setup(i)* para cada luz antes de desenhar a cena. A classe *Model* foi estendida com:

- `std::vector<UV> uvs`: coordenadas de textura por vértice;
- `Material material`: propriedades de cor e brilho;

- `unsigned int _texture`: identificador da textura OpenGL;
- método `loadTexture()`: carrega a imagem para o GPU via *stb_image*.

O `SceneParser` foi atualizado para ler os novos elementos `<lights>`, `<texture>` e `<color>` do XML, populando os campos correspondentes em `World` e `Model`.

O VBO passou a armazenar, por vértice, oito valores de ponto flutuante: três de posição, três de normal e dois de coordenada de textura, com a organização descrita na Tabela 1.

px	py	pz	nx	ny	nz	u	v
float	float	float	float	float	float	float	float

Tabela 1: Organização de um vértice no VBO (8 × float, 32 bytes)

4. Ficheiro .3d

O formato `.3d` mantém a estrutura da fase anterior mas cada linha de vértice passa a conter oito valores:

```
<n_vertices>
px py pz nx ny nz u v
...
```

onde (p_x, p_y, p_z) é a posição, (n_x, n_y, n_z) é a normal unitária e (u, v) são as coordenadas de textura em $[0, 1]^2$. O número inteiro na primeira linha indica o total de vértices.

A classe `Model` lê estes oito valores por vértice em `loadFromFile()` e constrói, em `setup()`, o VBO indexado eliminando vértices duplicados com base numa chave que compara os oito campos simultaneamente. Dois vértices são considerados distintos se diferirem em qualquer um dos campos, o que preserva descontinuidades de normal e de textura.

5. Generator

5.1. Plano

Um plano é subdividido numa grelha regular de $\text{divisions} \times \text{divisions}$ quadriláteros, cada um dividido em dois triângulos. Como a superfície é plana, todos os vértices partilham a mesma normal, calculada uma única vez como o produto externo normalizado dos dois vetores diretores u e v :

$$\hat{n} = \frac{\mathbf{u} \times \mathbf{v}}{|\mathbf{u} \times \mathbf{v}|} \quad (1)$$

As coordenadas de textura são obtidas normalizando a distância ao longo de cada eixo para $[0, 1]$. Para uma célula na posição (i, j) da grelha, com passo $\text{increment} = \text{length} / \text{divisions}$:

$$u = \frac{i \times \text{increment}}{\text{length}}, \quad v = \frac{j \times \text{increment}}{\text{length}} \quad (2)$$

Os quatro cantos de cada célula recebem assim as coordenadas UV (u, v) , $(u + 1/\text{divisions}, v)$, $(u, v + 1/\text{divisions})$ e $(u + 1/\text{divisions}, v + 1/\text{divisions})$.

5.2. Cubo

Um cubo é construído a partir de seis faces planas independentes. Cada face reutiliza a construção do plano com vetores diretores u e v ortogonais, escolhidos de modo a que $u \times v$ aponte sempre para o exterior. O cálculo das normais e das coordenadas UV é portanto idêntico ao do plano.

5.3. Esfera

A esfera é parametrizada pelo ângulo polar $\theta \in [0, \pi]$ (associado aos *stacks*) e pelo ângulo azimutal $\varphi \in [0, 2\pi]$ (associado aos *slices*):

$$p(\theta, \varphi) = \text{radius} \cdot (\sin \theta \sin \varphi, \cos \theta, \sin \theta \cos \varphi) \quad (3)$$

Como cada ponto da superfície se encontra à distância *radius* da origem, a normal unitária exterior é simplesmente o vetor posição normalizado:

$$\hat{n} = \frac{p}{|p|} = \frac{p}{\text{radius}} \quad (4)$$

Não é necessário calcular nenhum produto externo.

Para as coordenadas de textura os ângulos são mapeados linearmente para $[0, 1]^2$ através de uma projeção equiretangular (latitude-longitude):

$$u = \frac{\varphi}{2\pi}, \quad v = \frac{\theta}{\pi} \quad (5)$$

5.4. Cone

O cone tem duas regiões distintas: a base circular e a superfície lateral.

5.4.1. Base

A base está orientada para baixo, pelo que a sua normal é constante:

$$\hat{n}_{\text{base}} = (0, -1, 0) \quad (6)$$

As coordenadas UV colocam o centro da base em $(0.5, 0.5)$ e mapeiam os pontos do bordo radialmente num disco:

$$u = 0.5 + 0.5 \cos \theta, \quad v = 0.5 + 0.5 \sin \theta \quad (7)$$

5.4.2. Superfície Lateral

Um ponto no *stack* $i \in \{0, \dots, \text{stacks} - 1\}$ e no *slice* j é parametrizado pela fração de altura $t_i = i / \text{stacks}$:

$$\text{rBottom}(i) = (1 - t_i) \cdot \text{radius}, \quad h(i) = t_i \cdot \text{height}, \quad \theta_j = \frac{2\pi j}{\text{slices}} \quad (8)$$

$$p(i, j) = (\text{rBottom}(i) \cos \theta_j, h(i), \text{rBottom}(i) \sin \theta_j) \quad (9)$$

Normais. A normal lateral tem de estar inclinada para o exterior de acordo com o ângulo de semi-abertura do cone. Com comprimento de geratriz $L = \sqrt{\text{radius}^2 + \text{height}^2}$, a normal unitária exata no ângulo θ_j é:

$$\hat{n} = \frac{1}{L}(\text{height} \sin \theta_j, \text{radius}, \text{height} \cos \theta_j) \quad (10)$$

Na implementação, isto é expresso proporcionalmente como

$$\mathbf{n}_{\text{raw}} = (\sin \theta_j, \text{rBottom}(i)/\text{height}, \cos \theta_j) \quad (11)$$

e depois normalizado, obtendo a mesma direção.

Coordenadas de Textura. A superfície lateral é desdobrada cilindricamente:

$$u = \frac{\theta_j}{2\pi}, \quad v = t_i = \frac{i}{\text{stacks}} \quad (12)$$

5.5. Cilindro

Um cilindro tem três regiões: duas tampas planas e uma banda lateral.

5.5.1. Tampas

Ambas as tampas têm normais constantes alinhadas com o eixo y :

$$\hat{n}_{\text{top}} = (0, 1, 0), \quad \hat{n}_{\text{bot}} = (0, -1, 0) \quad (13)$$

As coordenadas UV mapeiam o bordo num disco, de forma idêntica à base do cone:

$$u = 0.5 + 0.5 \cos \alpha, \quad v = 0.5 + 0.5 \sin \alpha \quad (14)$$

onde $\alpha = i \cdot 2\pi / \text{slices}$ e $\text{nextAlpha} = (i + 1) \cdot 2\pi / \text{slices}$.

5.5.2. Superfície Lateral

Como o cilindro não tem inclinação, a normal exterior no azimute α é puramente radial:

$$\hat{n} = (\cos \alpha, 0, \sin \alpha) \quad (15)$$

O desdobramento UV atribui $v = 0$ ao bordo inferior e $v = 1$ ao bordo superior:

$$u = \frac{\alpha}{2\pi}, \quad v \in \{0, 1\} \quad (16)$$

5.6. Anel

O anel é parametrizado pelo ângulo maior $\theta \in [0, 2\pi]$ (*rings*) e pelo ângulo do tubo $\varphi \in [0, 2\pi]$ (*sides*), com raio maior `majorRadius` e raio do tubo `minorRadius`:

$$\begin{aligned} p(\theta, \varphi) = (& \\ & (\text{majorRadius} + \text{minorRadius} \cos \varphi) \cos \theta, \\ & \text{minorRadius} \sin \varphi, \\ & (\text{majorRadius} + \text{minorRadius} \cos \varphi) \sin \theta \\ &) \end{aligned} \quad (17)$$

A normal unitária exterior aponta para fora do eixo do tubo:

$$\hat{n} = (\cos \varphi \cos \theta, \sin \varphi, \cos \varphi \sin \theta) \quad (18)$$

Este vetor é já unitário, dado que $\cos^2 \varphi + \sin^2 \varphi = 1$.

Para as coordenadas de textura, ambos os ângulos são mapeados linearmente para $[0, 1]^2$:

$$u = \frac{\theta}{2\pi}, \quad v = \frac{\varphi}{2\pi} \quad (19)$$

5.7. Bezier Patches

Um patch de Bézier é definido por 16 pontos de controlo P_{ij} , com $i, j \in \{0, 1, 2, 3\}$, e parâmetros $u, v \in [0, 1]$.

$$S(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_i(u) B_j(v) P_{ij} \quad (20)$$

onde $B_k(t)$ são os polinómios de Bernstein cúbicos:

$$\begin{aligned} B_0(t) &= (1-t)^3 \\ B_1(t) &= 3t(1-t)^2 \\ B_2(t) &= 3t^2(1-t) \\ B_3(t) &= t^3 \end{aligned} \quad (21)$$

A normal é obtida pelo produto externo dos dois vetores tangentes parciais. O tangente na direção u é:

$$T_u = \frac{\partial S}{\partial u} = \sum_{i=0}^3 \sum_{j=0}^3 B'_i(u) B_j(v) P_{ij} \quad (22)$$

e na direção v :

$$T_v = \frac{\partial S}{\partial v} = \sum_{i=0}^3 \sum_{j=0}^3 B_i(u) B'_j(v) P_{ij} \quad (23)$$

As derivadas dos polinómios de Bernstein são:

$$\begin{aligned} B'_0(t) &= -3(1-t)^2 \\ B'_1(t) &= 3(1-t)^2 - 6t(1-t) \\ B'_2(t) &= 6t(1-t) - 3t^2 \\ B'_3(t) &= 3t^2 \end{aligned} \quad (24)$$

A normal unitária exterior é então:

$$\hat{n} = \frac{T_v \times T_u}{|T_v \times T_u|} \quad (25)$$

Quando o *patch* degenera – i.e., quando $|T_v \times T_u| < 10^{-6}$, utiliza-se a normal de recurso $(0, 1, 0)$.

As coordenadas paramétricas são usadas diretamente como coordenadas UV. Para a célula no índice de tesselação (u_i, v_i) com nível tessellation:

$$u = \frac{u_i}{\text{tessellation}}, \quad v = \frac{v_i}{\text{tessellation}} \quad (26)$$

A textura é assim esticada uniformemente sobre cada *patch*, sem qualquer correção de distorção adicional.

6. Engine

6.1. Iluminação

O sistema de iluminação usa o modelo de iluminação de Phong disponível no *pipeline* fixo do OpenGL. São suportados três tipos de fonte de luz, configuráveis via XML:

- **Pontual** (*point*): emite luz em todas as direções a partir de uma posição. Configurada com `GL_POSITION` com componente $w = 1$.
- **Direcional** (*directional*): simula uma fonte à distância infinita, com raios paralelos numa direção fixa. Configurada com `GL_POSITION` com componente $w = 0$.
- **Foco** (*spot*): emite um cone de luz com ângulo de abertura `cutoff` e concentração `exponent`. Configurada com `GL_SPOT_DIRECTION`, `GL_SPOT_CUTOFF` e `GL_SPOT_EXPONENT`.

A inicialização da iluminação ocorre uma única vez em `World::setup()`, que chama `Light::enableLighting()`. Este método ativa `GL_LIGHTING`, `GL_NORMALIZE` (para renormalizar as normais após escalonamento) e define `GL_LIGHT_MODEL_TWO_SIDE` para iluminar ambas as faces dos triângulos.

Em cada *frame*, antes de qualquer transformação de modelo, `World::draw()` itera sobre as luzes e chama `Light::setup(i)` para cada uma, registrando a posição e direção no espaço do mundo.

O XML aceita o seguinte formato para declarar luzes:

```
<lights>
  <light type="point"      posx="0"  posy="0"  posz="0" />
  <light type="directional" dirx="1"  diry="0"  dirz="0" />
  <light type="spot"       posx="0"  posy="5"  posz="0"
                           dirx="0"  diry="-1" dirz="0"
                           cutoff="30" exponent="2" />
</lights>
```

6.2. Materiais

Cada modelo pode ter um conjunto de propriedades de material definidas no XML através do elemento `<color>`. As componentes suportadas são:

- **Difusa** (*diffuse*): cor principal, modulada pelo ângulo de incidência da luz.
- **Ambiente** (*ambient*): contribuição constante, independente da orientação.
- **Especular** (*specular*): reflexo brilhante, dependente do ângulo de visualização.
- **Emissiva** (*emissive*): luz própria do objeto, independente das fontes externas.
- **Brilho** (*shininess*): expoente que controla a concentração do reflexo especular.

Os valores RGB são dados em $[0, 255]$ no XML e convertidos para $[0, 1]$ pelo *parser*. Em `Model::draw()`, as propriedades são aplicadas via `glMaterialfv` antes de cada `glDrawElements`, afetando apenas o modelo a que pertencem.


```

<color>
  <diffuse R="169" G="169" B="169" />
  <ambient R="30" G="30" B="30" />
  <specular R="80" G="80" B="80" />
  <emissive R="0" G="0" B="0" />
  <shininess value="20" />
</color>

```

6.3. Texturas

O carregamento de texturas é feito com a biblioteca *stb_image*, integrada diretamente no projeto em `lib/stb_image/`. O método `Model::loadTexture()` é chamado dentro de `Model::setup()`, após o contexto OpenGL estar disponível, e executa os seguintes passos:

1. Lê a imagem com `stbi_load`, forçando três canais (RGB) e invertendo verticalmente para compensar a diferença de origem entre *stb_image* e OpenGL.
2. Cria um objeto de textura com `glGenTextures` e carrega os dados para a GPU via `glTexImage2D`.
3. Gera *mipmaps* automáticos com `glGenerateMipmap` e configura filtragem linear (`GL_LINEAR`) e repetição (`GL_REPEAT`).

Durante o desenho, se o modelo tiver textura, `Model::draw()` ativa `GL_TEXTURE_2D`, associa a textura com `glBindTexture` e define o modo `GL_MODULATE`, que combina a cor da textura com a contribuição do material. O VBO inclui as coordenadas UV e o *pointer* `GL_TEXTURE_COORD_ARRAY` aponta para o deslocamento correto dentro do mesmo *buffer* interleaved.

A textura é declarada no XML através de:

```

<model file="sphere.3d">
  <texture file="textures/earth.jpg" />
</model>

```

7. Demo Scene

A cena de demonstração representa o Sistema Solar e aplica todas as funcionalidades introduzidas nesta fase.

Iluminação. Uma luz pontual é colocada na origem, representando o Sol. Desta forma, os planetas recebem luz de acordo com a sua orientação face ao Sol, com as faces voltadas para ele mais iluminadas.

Sol. Representado por uma esfera escalonada ($\times 3$) com componente emissiva ativa (amarelo-laranja), tornando-o visível mesmo sem fontes de luz externas.

Planetas. Todos os planetas possuem textura e material configurados individualmente. As propriedades de material foram escolhidas para refletir as características visuais de cada corpo celeste (e.g., Neptuno tem componente especular azul intensa; Mercúrio tem especular reduzida). As órbitas são animadas com `<rotate time=" ... ">`.

Satélites. A Lua orbita a Terra, Fobos e Deimos orbitam Marte, e Ganimedes orbita Júpiter. Todos herdam a translação do planeta-pai através da hierarquia de grupos, tendo depois as suas próprias rotações no referencial local.

Saturno. Além da textura no planeta, foi adicionado um anel (`saturn_ring.3d`) com rotação estática inclinada e material próprio.

Bule. O *teapot* orbita numa curva de *Catmull-Rom* tridimensional com `align="true"`, mantendo a orientação alinhada com a tangente da trajetória. Tem textura e material especular elevado (`shininess = 128`).

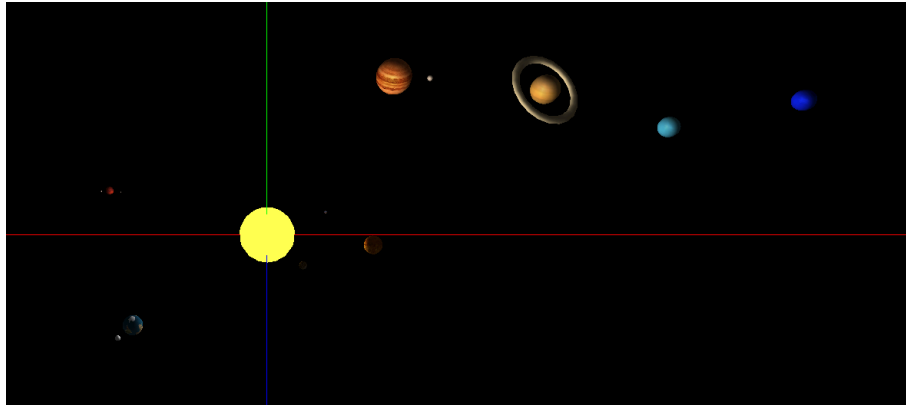


Figura 2: Sistema Solar com iluminação e texturas

8. Conclusão

Esta fase completou o sistema com iluminação, materiais e texturas. O *generator* foi atualizado para exportar normais e coordenadas UV para todas as primitivas. O formato `.3d` passou a incluir esses dados, e o VBO foi reorganizado para os conter de forma *interleaved*. O *engine* passou a configurar luzes OpenGL (pontuais, direcionais e focos), a aplicar materiais por modelo via `glMaterialfv` e a carregar texturas com `stb_image`. A *demo scene* do Sistema Solar demonstra todas estas funcionalidades em conjunto: a luz pontual central ilumina os planetas de forma realista, as texturas individualizam cada corpo celeste, e os materiais controlam o brilho e as reflexões especulares de cada objeto.